

02 프레임워크 개발자 가이드

CPF 를 처음 사용하는 Java 개발자가 기능을 찾고 선택해 업무 API·DB·Transaction·연계 기능을 개발·검증할 수 있도록 구성한 실무 가이드입니다.

CPF 공식 문서 · 기준 Source bd5fcd7707cf · 2026-08-15

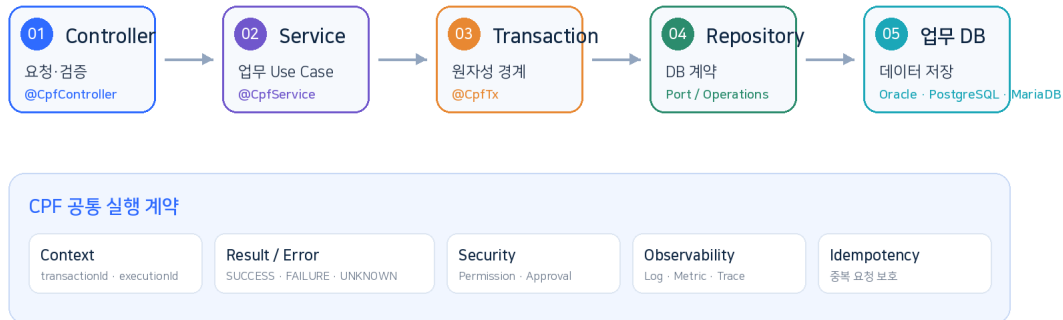
사용 방식 먼저 “하려는 일”로 기능을 찾고, 공개 API·메서드·Annotation 과 선택 기준을 확인합니다. Source 예제는 계약을 설명하는 데 필요한 경우만 사용하며 Framework 내부 구현 분석을 전제로 하지 않습니다.

1. 개발 작업 빠른 찾기

하려는 일	시작 API/기능	핵심 판단	상세
REST API 작성	@CpfController / CpfBaseController	Validation·Permission·Error	§5
업무 Service 작성	@CpfService / CpfBaseService	@CpfTx·Audit·Idempotency	§6
DB 조회·CRUD	Repository / CpfDataOperations	Paging·Lock·Provider	§7
조회 전용 TX	@CpfTx(readOnly=true)	Manager·Timeout	§8
독립 TX	@CpfTx(REQUIRES_NEW)	별도 Bean·self-call 주의	§8.3
다른 Domain 호출	Generated Typed Client.execute()	AUTO/LOCAL/REMOTE·CpfResult	§9
외부 시스템 호출	@CpfClient / @CpfTimeout / @CpfRetry	Side Effect·UNKNOWN	§10
중복 요청 보호	@CpfIdempotent	Key·Payload·TTL·동시 호출	§11
권한/승인/감사	@CpfPermission / @CpfApprovalRequired / @CpfAudit	Reason·Audit	§12
비동기/Event	Messaging / Outbox·Inbox 실제 계약	TX 확정 시점·중복	§13
Cache/Lock	Cache / Distributed Lock Starter	일관성·TTL·Stampede	§14
생성·DB Migration	Generator / DB Lifecycle	dry-run·diff·3 Vendor	§15

Online 거래 개발 흐름

업무 개발자는 Controller → Service → Repository 계약을 사용하고, CPF가 Context·Transaction·오류·관측을 연결합니다.



핵심: 업무 로직은 Service에 두고, Remote/Async 호출은 Local DB Transaction과 별도 경계로 설계합니다.

Online 거래의 대표 흐름

2. CPF 개발 모델

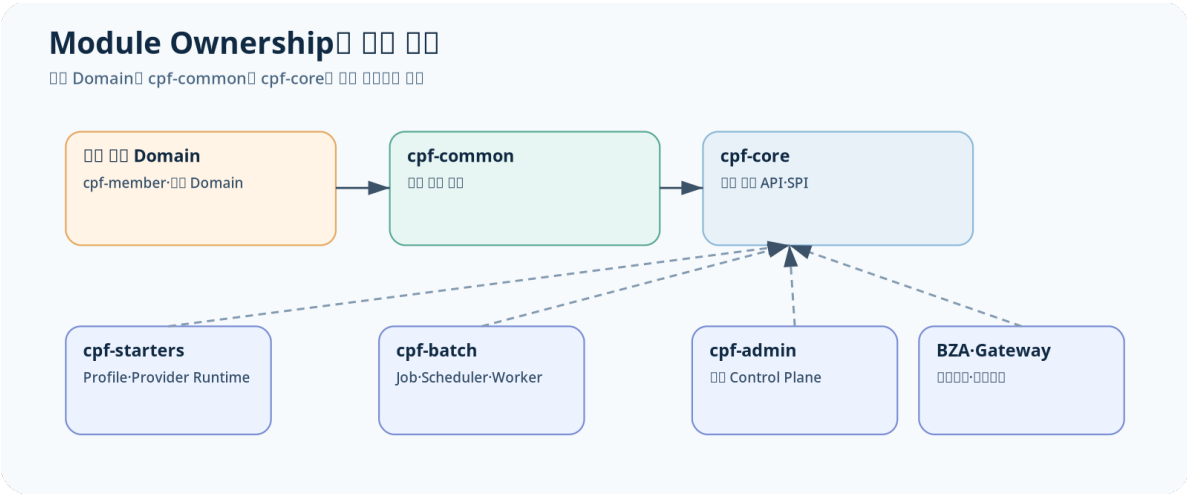
업무 개발은 Generated Domain 에서 수행합니다. Controller·Service·Persistence 의 3 단 Base 구조를 사용하고, 업무 Source 는 CPF 의 Public API 와 Public Starter/Profile 을 통해 공통 기능을 사용합니다.

계층	구조	개발자 책임
Controller	CpfBaseController → DomainBaseController → BusinessController	HTTP 계약·Validation·Response
Service	CpfBaseService → DomainBaseService → BusinessService	Use Case·Transaction·정책 조합
Persistence	CpfBaseDao → DomainBaseDao → BusinessDao 또는 Repository Port	DB 접근·Paging·Lock
Domain Call	Typed Client → Runtime Router	다른 Domain Contract 호출
Async	Event/Message 계약 → Consumer	중복·순서·실패·복구

패키지 고정 `customer` 중간 Package 를 요구하지 않습니다. 조직 Base Package 다음에 Domain 이 바로 이어지는 구조를 사용합니다.
예: `com.acme.member`.

3. Module·Dependency 선택

목적	Owner/구성	직접 사용 기준
핵심 계약	cpf-core	Public 계약만 사용; 업무 코드 적치 금지
Web/API	cpf-starters/web	Controller·Web 공통 기능
DB	cpf-starters/data/persistence	JDBC/MyBatis/JPA 등 선택 Provider
Transaction	Persistence Starter	@CpfTx Golden Path
외부 연계	Public Integration Profile	내부 leaf 직접 참조 금지
Messaging	Messaging Starter	Broker Provider 는 Profile/설정으로 선택
Security	Security Starter	Permission·Approval·Session 등
Batch	cpf-batch + Batch Starter	03 배치 개발자 가이드
Gateway	cpf-gateway	외부 진입 정책이 필요할 때 선택



업무 Domain 은 Owner Internal 구현보다 Public 계약을 사용

4. 개발 명령과 Generator

목적	실행 위치	명령/기능	결과 확인
Repository 구조	Root	<code>Wgradlew.bat projects</code>	Module 포함 확인
전체 Test	Root	<code>Wgradlew.bat test</code>	실패 Module/Test 확인
Domain 정의 검증	Root	<code>Wcpf-toolsWruntimeWcliWcpf.bat domain validate --file <cpf-domain.yaml></code>	정의·사전 조건 확인
영향 확인	Root	<code>Wcpf-toolsWruntimeWcliWcpf.bat domain dry-run --file <cpf-domain.yaml></code>	생성/변경 예상 확인
최초 생성	Root	<code>Wcpf-toolsWruntimeWcliWcpf.bat domain generate --file <cpf-domain.yaml></code>	Generated Domain 생성

목적	실행 위치	명령/기능	결과 확인
차이 확인	Root	<code>`.Wcpf-toolsWruntimeWcliWcpf.bat domain diff --file <cpf-domain.yaml> --output <cpf-domain>`</code>	정보 대비 변경
재생성/Upgrade	Root	<code>`.Wcpf-toolsWruntimeWcliWcpf.bat domain regenerate <domain>` / `domain upgrade <domain>`</code>	소유 파일·호환성 확인

명령 사실성 Repository의 실제 CLI/Gradle Task와 일치하는 명령만 사용합니다. Generator 세부 Syntax는 현재 Generator 계약을 기준으로 확인하고, 존재하지 않는 편의 명령을 문서에 만들지 않습니다.

5. Controller 공통 기능

하려는 일	CpfBaseController 메서드	특징
현재 Context	<code>requireCurrentContext()</code>	현재 실행 문맥 필수
불변 Snapshot	<code>requireContext()</code>	하위 계층 전달용
문자열 검증	<code>requireText(value, fieldName)</code>	null/blank 거부
필수 값	<code>requireValue(value, fieldName)</code>	null 거부
규칙 검증	<code>requireRule(condition, message)</code>	표준 Validation 예외
200 응답	<code>ok(body)</code>	표준 성공 응답
201 응답	<code>created(location, body)</code>	Location 검증 포함
204 응답	<code>noContent()</code>	Body 없음
추적 정보	<code>executionFacts(operation)</code>	transactionId/executionId/actorId/tenantId

6. Service·DAO 공통 기능

계층	공통 메서드	사용 기준
CpfBaseService	<code>requireText(value, fieldName)</code>	Framework Base는 공통 문자열 필수 검증만 제공
CpfBaseDao	<code>requireCurrentContext()</code> , <code>requireContext()</code>	DB 작업과 실행 식별자 연결
CpfBaseDao	<code>statementId(namespace, statement)</code>	Named Statement 안전 조합
CpfBaseDao	<code>boundedSize(requested, default, max)</code>	Page/Fetch 상한
CpfBaseDao	<code>executionFacts(operation)</code>	비민감 DB 실행 추적

Domain Base Service Generator가 만드는 Domain Base Service는 Framework Base 위에 ``requireTransactionId()``, ``transactionSequence()``, ``actorId()``처럼 해당 Domain에서 반복되는 정책을 추가합니다. Framework Base와 Domain Base의 책임을 섞지 않습니다.

7. Persistence 와 조회 방식

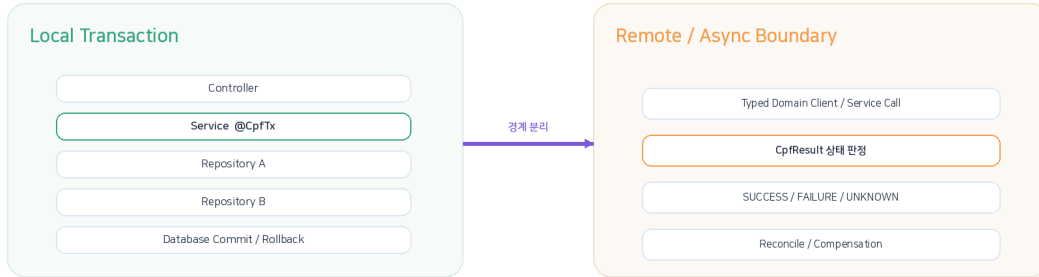
처리 목적	공개 계약	선택 기준
Named 목록	CpfDataOperations.selectList(...)	Provider-neutral Operation
Named 단건	selectOne(...)	0/1 건 계약
등록·수정·삭제	insert(...) / update(...) / delete(...)	영향 건수·Version·삭제 정책 확인
Rollback 검증	inRollbackOnlyTransaction(...)	항상 Rollback 되는 검증 경계
JDBC 조회	CpfJdbcOperations.queryList(...) / queryOne(...)	Infrastructure Adapter 내부·Vendor SQL 관리
JDBC 변경	CpfJdbcOperations.update(...)	업무 Source 에 SQL Literal 분산 금지

목록 방식	적합한 상황	주의
Page + Total	전체 건수 필요	Count 비용
Slice	다음 페이지 여부만 필요	Total 없음
Cursor/Keyset	대용량·변경 중 탐색	안정된 Sort Key
List	작은 고정 범위	상한 없는 전체 조회 금지

8. Transaction

Transaction 경계 선택

Local DB 원자성과 Remote/Async 결과 확정은 같은 문제로 보지 않습니다.



독립 처리: @CpFTx(... propagation = REQUIRES_NEW)
감시/후속 처리를 주 거래와 독립 Commit해야 할 때 별도 Bean 경계에서 사용

Local Transaction 과 Remote/Async 경계를 분리

@CpFTx 속성	기본값	사용 의미
id	필수	안정된 Transaction 식별자
name	필수	사람이 이해하는 명칭
ownerDomain	필수	소유 Domain
transactionManager	""	다중 DataSource Manager 선택
propagation	REQUIRED	Transaction 참여/생성 방식
isolation	DEFAULT	DB Isolation
requiredPermission	""	필요 Permission
auditReasonRequired	false	사유 필수
visibility	PUBLIC	노출 정책
gatewayAllowed	true	Gateway 진입 허용
directAllowed	false	Direct 진입 허용
readOnly	false	조회 전용
timeoutSeconds	-1	Provider 기본 Timeout 사용

8.1 Propagation 선택

방식	사용 시점	핵심 특성
REQUIRED	일반 업무	기존 TX 참여, 없으면 생성
REQUIRES_NEW	독립 Commit 필요	기존 TX 일시 중단 후 새 TX
SUPPORTS	TX 선택적 보조 처리	없어도 실행
MANDATORY	상위 TX 필수	없으면 오류
NOT_SUPPORTED	반드시 TX 밖	기존 TX 일시 중단
NEVER	TX 존재 금지	TX 있으면 오류
NESTED	Savepoint 기반 부분 Rollback	Provider 지원 확인

8.2 독립 Transaction - REQUIRES_NEW

주 업무 Rollback 과 무관하게 감사·보정 요청 등 독립 보존이 필요한 경우 사용합니다. 동일 객체 내부 호출은 Proxy 경계를 우회할 수 있으므로 독립 Transaction 메서드는 별도 Bean 경계로 분리합니다.

주 TX	독립 TX	결과
Commit	Commit	둘 다 보존
Rollback	Commit	독립 처리만 보존
Commit	Rollback	주 업무 보존, 독립 처리 실패 별도 대응
Rollback	Rollback	둘 다 취소

8.3 Transaction 과 다른 기능 조합

조합	확인할 계약
TX + Retry	Retry 마다 동일/신규 TX 인지, 멍등성
TX + Async/Event	Thread 변경·Commit 전후 Event 확정
TX + Cache	Commit 이전 Cache 변경 여부
TX + Remote	Remote 결과를 Local 원자성으로 가정하지 않음
TX + Batch	Chunk/Step 단위 Commit
TX + REQUIRES_NEW	독립 Commit 실패/성공의 최종 결과

9. Domain 간 호출

업무 개발자의 1 차 진입점은 Generator 가 만든 Typed Domain Client 의 `execute(request)` 입니다.
`CpfDomainClientRouter.invoke()` 는 Local/Remote Binding 을 해석하는 Runtime 내부 계층이며 일반 업무 Service 가 직접 호출하는 Golden Path 가 아닙니다.

개발 단계	사용 계약	지원/판단
Client 의존	...DomainClient extends CpfDomainClient	업무 Service 가 Typed Client 주입
호출	client.execute(request)	CpfResult<Response> 반환
Binding	AUTO / LOCAL / REMOTE	배포 위치에 따라 선택
상태 확인	isSuccess / isBusinessFailure / isTechnicalFailure / isUnknown	4 상태 분기
함수형 처리	fold(...)	상태별 후처리
데이터 사용	requireData()	SUCCESS 확정일 때만

Binding	적용 기준	주의
LOCAL	같은 Runtime 에 Operation 존재	미등록 시 기술 실패
REMOTE	분리 Runtime	Timeout·인증·직렬화·UNKNOWN
AUTO	Local 가능 시 Local, 아니면 Remote	Remote serviceld 등 필수 계약

10. 외부 시스템 연계

기능	개발 계약	핵심 옵션/판단
Client	@CpfClient	system·operation·sideEffecting·contextRequired
Timeout	@CpfTimeout	millis; 전체/시도 Budget
Retry	@CpfRetry	maxAttempts·delayMillis·reconcileUnknownOutcome
결과	CpfResult	SUCCESS/업무실패/기술실패/UNKNOWN
고정길이/TCP/SOAP	Public Integration Profile 을 통해 사용	내부 Leaf 직접 의존 금지
파일/SFTP/Object	해당 Public Capability	부분 실패·Checksum·재처리 확인

Side Effect Timeout 상대 시스템이 처리했는데 응답만 유실될 수 있습니다. 상태 변경 요청 Timeout 을 단순 실패로 간주해 즉시 새 요청으로 재전송하지 말고 Idempotency·Status Query·Reconcile 계약을 먼저 확인합니다.

11. Idempotency·Concurrency

상황	기능	판단 기준
동일 요청 재전송	@CpfIdempotent	Key/Payload 동일성·처리 중·완료·TTL
화면 동시 수정	Optimistic Version	최신 Version 재조회
선점 처리	DB Lock / 분산 Lock	Lock Timeout·Deadlock
중복 데이터	Unique Constraint	업무 Key 를 DB 에서도 보호
Remote 재전송	Idempotency Key + Ledger/Status	UNKNOWN 에서 새 Key 금지

12. Security·Approval·Audit

목적	계약	개발 확인
권한	@CpfPermission	Authority fail-closed
승인	@CpfApprovalRequired	Action·승인 수·Reason·상태
감사	@CpfAudit	Actor·Action·Target·Result·비민감 Summary
Data Scope	Security/공통 계약	조회/변경 범위
Masking	Logging/UI/Export 정책	민감정보 원문 노출 금지
Session/Identity	선택 Session/Service Identity	Runtime/배포 형태에 맞게 선택

13. Messaging·Event

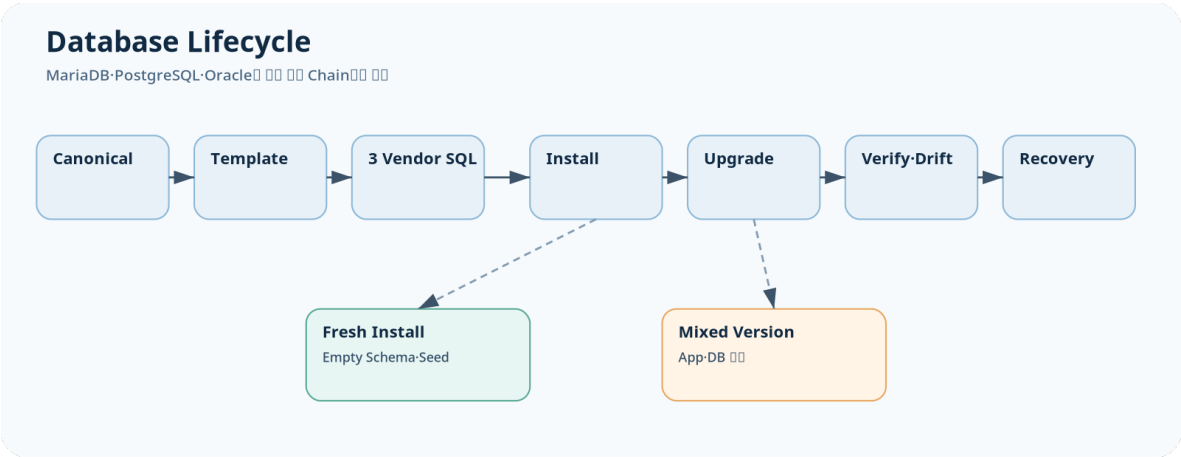
하려는 일	적용 방식	반드시 확인
업무 Event 발행	Messaging/Outbox 실제 계약	DB Commit 과 Event 확정 시점
Consumer 처리	@CpfMessageListener	consumerGroup·idempotencyRequired·context Required
중복 Event	Inbox/Idempotency	같은 Event 재수신 허용 여부
실패 재처리	Retry/DLQ/Reprocess 실제 구현	업무 Side Effect 중복
Ordering	Topic/Key 정책	필요한 Ordering 범위

14. Cache·Lock·공통 Service

목적	기능군	선택/주의
반복 조회	Cache	TTL·Invalidation·Serialization
다중 Instance Cache	Redis/Valkey Provider	Reconnect·Stale·Provider 선택
Local Cache	Caffeine 계열	Instance 별 불일치 허용 여부
동시 처리 보호	Distributed Lock	TTL·Owner·Timeout·Fencing 필요성

목적	기능군	선택/주의
기준정보·Template	Code/Parameter/Message/Calendar/Template	cpf-common 소유 공통 기능; 업무 중복 방지

15. Generator·DB Lifecycle



Canonical DB에서 공식 3 Vendor 산출물과 Migration을 관리

단계	개발자 확인
정의/검증	Domain·기능·DB 선택과 Naming 검증
dry-run	생성/변경 영향 확인
generate	업무 Source 생성
diff/regenerate	개발자 수정 영역과 Generator 소유 영역 구분
upgrade	Template/계약 Version 전환
DB Install/Migration	Oracle/PostgreSQL/MariaDB 공식 Vendor
Seed/Verify/Rollback	중복·Checksum·되돌림 영향 확인

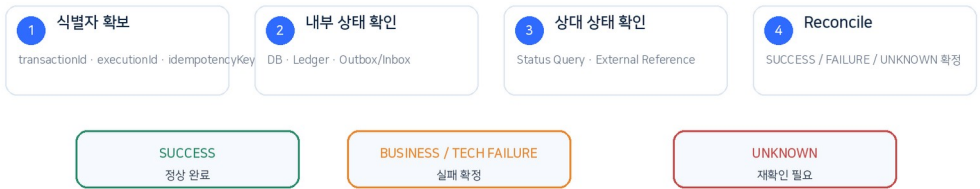
16. Error·UNKNOWN·Recovery

UNKNOWN_RESULT 판단 흐름

Timeout이나 응답 유실은 곧바로 실패를 뜻하지 않습니다. 실제 처리 여부를 먼저 확정합니다.



확인 순서



금지: 결과가 미확정인 상태에서 새 Idempotency Key로 동일 Side Effect 요청을 즉시 재실행하지 않습니다.

UNKNOWN은 실패가 아니라 실제 처리 여부가 확정되지 않은 상태

상태	의미	개발자 기본 행동
SUCCESS	성공 확정	결과 사용
BUSINESS_FAILURE	업무 거절 확정	입력/상태 수정, 일반적으로 재시도하지 않음
TECHNICAL_FAILURE	기술 실패 확정	원인 제거 후 멍등성 조건에서 Retry 판단
UNKNOWN	처리 여부 미확정	Status/Reconcile/Compensation 우선

복구 방식	대상	사용 시점
Retry	동일 Operation 재시도	일시 오류 + 멍등성 보장
Restart	중단된 실행의 Checkpoint 재개	Batch 등 실행 상태가 존재
Reprocess	선택 실패 업무 건 신규 처리	업무 건 단위 재처리
Reconcile	시스템 간 실제 결과 확정	UNKNOWN/부분 실패
Compensation	이미 확정된 Side Effect 보정	Rollback 로 되돌릴 수 없는 분산 처리
Rollback	Local TX/배포/설정 등 원복	경계가 명확한 변경

17. Test·검증

검증 유형	필수 확인
정상	대표 요청/응답·DB
Validation	필수/경계/형식
Transaction	Commit/Rollback/REQUIRES_NEW
Concurrency	Version/Lock 충돌
Duplicate	동일 Key 재전송
Remote	Timeout/Connection Reset/UNKNOWN
Recovery	Retry/Reprocess/Reconcile/Compensation 중 해당
Security	401/403/Permission/Approval/Audit
운영 연결	transactionId/tracelId 로 ADM·Log 확인

18. 공개 API·Annotation 빠른 참조

개발 목적	공개 계약	대표 메서드/속성
HTTP	@CpfController / CpfBaseController	ok·created·noContent·require*
Service	@CpfService / CpfBaseService	requireText; Domain Base 가 거래 ID·Actor 정책 추가
Persistence	@CpfRepository / @CpfDao / CpfDataOperations	selectList·selectOne·insert·update·delete
Transaction	@CpfTx	manager·propagation·isolation·readOnly·timeout
Domain Call	CpfDomainClient	execute
Result	CpfResult	isSuccess·isBusinessFailure·isTechnicalFailure·isUnknown·fold·requireData
External	@CpfClient / @CpfTimeout / @CpfRetry	system·operation·timeout·attempt
Idempotency	@CpfIdempotent	operation·ttl·InProgressTimeout·replayResult
Messaging	@CpfMessageListener	destination·consumerGroup·idempotencyRequired
Security	@CpfPermission / @CpfApprovalRequired	permission·approval·reason
Audit	@CpfAudit	action·reasonRequired·safe summary

19. Source Navigation

목적/계약	정본 Source
CpfBaseController	cpf-starters/web/src/main/java/com/cpf/web/api/CpfBaseController.java
CpfBaseService	cpf-starters/base/runtime/src/main/java/com/cpf/foundation/api/CpfBaseService.java
CpfBaseDao	cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/CpfBaseDao.java

목적/계약	정본 Source
CpfTx	cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/annotation/CpfTx.java
CpfDomainClient	cpf-core/src/main/java/com/cpf/core/api/domain/CpfDomainClient.java
CpfResult	cpf-core/src/main/java/com/cpf/core/api/result/CpfResult.java
CpfDataOperations	cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/database/CpfDataOperations.java
CpfJdbcOperations	cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/database/CpfJdbcOperations.java
Generator	cpf-tools/generator/**
DB Lifecycle	cpf-tools/db/**
Education	cpf-education/**

20. 개발 완료 Checklist

영역	완료 조건
Architecture	올바른 Owner/Layer, Internal 직접 참조 없음
API	Validation·Error·Permission 계약
Transaction	Boundary·Rollback·REQUIRES_NEW·Remote 경계
Concurrency	Version/Lock/중복 판단
Idempotency	Key·Payload 동일성·TTL·재전송 정책
Security	Permission·Data Scope·Masking·Approval·Audit
DB	Migration·3 Vendor 영향·Rollback
Messaging/Event	Commit 시점·중복 Consumer·재처리
Cache/Lock	TTL·Invalidation·다중 Instance·Lock Timeout
Async/Remote	Timeout·Idempotency·UNKNOWN
Observability	transactionId/tracelId·Log·Metric·ADM 연결
Generator	validate·dry-run·diff·regenerate/upgrade 영향 확인
Test	정상·오류·경계·동시성·복구
운영 인계	Config·Migration·Permission·Monitoring·Rollback
Specification	사용 계약·Default·계약이 07 기술 명세와 일치